Mini POS v1.0.0 Stable

Architecture Specification & Commercial Setup Guide

Production Package
Flutter 3.x • SQLite Embedded • Offline-First

1. ARCHITECTURE & SOFTWARE DESIGN PATTERN

Mini POS is a standalone micro-enterprise management and point-of-sale system built on an **Offline-First Layered Architecture**. It operates without remote server dependencies, guaranteeing zero latency, zero cloud costs, and full operational capability in network-constrained environments.

- **Presentation Layer (`lib/screens` , `lib/widgets`):** Declarative Material 3 UI. Widgets remain decoupled from data operations, observing state streams provided by ChangeNotifier controllers.
- **State & Controller Layer (`lib/providers`):** Encapsulates screen-level logic, reactive UI updates, input validation, and transactional view-state management.
- **Domain & Repository Layer (`lib/repositories`):** Acts as a single source of truth for business operations, coordinating data access and entity transformations.
- **Data Access Layer (`lib/daos` , `lib/core`):** Direct parameterized SQLite operations utilizing ACID-compliant transactional execution via `sqflite` / `sqflite_common_ffi` .
- **Internationalization (`lib/l10n`):** Dynamic locale resolution supporting English (`app_en.arb`) and Portuguese (`app_pt.arb`) with compile-time type-safe string generation.

2. DATABASE SCHEMA & MIGRATION LIFECYCLE

The local SQLite database enforces relational integrity using foreign key constraints and indexed lookup queries. All schema evolutions (v1 through v4) are version-controlled inside `core/local_database.dart` :

```
// Database migration handler (core/local_database.dart)
Future<void> _onUpgrade(Database db, int oldVersion, int newVersion) async {
  if (oldVersion < 2) await _migrateToV2(db); // Multi-store tenant support
  if (oldVersion < 3) await _migrateToV3(db); // Split-payment & debt ledger
  if (oldVersion < 4) await _migrateToV4(db); // Statement snapshot indexing
}
```

3. CORE FUNCTIONAL CAPABILITIES

- **Multi-Branch Isolation:** Complete logical partition of customers, inventory, expenses, and transaction logs per business unit.
- **Credit & Debt Ledger:** Tracks partial installments, calculates balances dynamically, and automatically transitions fully-settled accounts.
- **Analytical Reporting:** Aggregated revenue metrics, store-by-store performance comparisons, and exportable financial summaries.
- **Tamper-Proof Backups:** Comprehensive JSON state exports with conflict-free timestamp reconciliation on data import.

4. DEVELOPMENT ENVIRONMENT & BUILD TOOLCHAIN

- **Flutter SDK:** Version `>=3.19.0 <4.0.0` (Stable Channel)
- **Dart SDK:** Version `>=3.3.0 <4.0.0`
- **Java Development Kit (JDK):** OpenJDK 17
- **Android Target SDK:** `compileSdk = 36` , `targetSdk = 36` , `minSdk = 21`

```
# Resolve dependencies & generate localization files
flutter pub get
flutter gen-l10n

# Launch application on connected emulator / hardware device
flutter run
```

5. REBRANDING & CUSTOMIZATION BLUEPRINT

A. Application ID & Namespace Configuration

Update the package namespace inside `android/app/build.gradle.kts` to your company's domain structure:

```
android {
    namespace = "com.yourcompany.minipos"
    defaultConfig {
        applicationId = "com.yourcompany.minipos"
        minSdk = 21
        targetSdk = 36
        versionCode = 1
        versionName = "1.0.0"
    }
}
```

B. Theme Tokens & Brand Palette

Modify primary accent colors and Material 3 seed palettes centrally in `lib/theme/app_theme.dart`:

```
static const Color seedColor = Color(0xFF1E40AF); // Primary Brand Color
static final ThemeData lightTheme = ThemeData(
  useMaterial3: true,
  colorScheme: ColorScheme.fromSeed(seedColor: seedColor, brightness: Brightness.light),
);
```

C. App Launcher Icons

Place your 512x512 PNG asset at `assets/icon.png` and execute the automated icon generator:

```
dart run flutter_launcher_icons
```

6. ANDROID STORAGE & SCOPED STORAGE COMPLIANCE

Google Play Policy Compliance:
The project isolates local database files inside internal app storage (`getDatabasesPath()`). Exported backups and financial statements utilize `path_provider` sandbox storage and dispatch externally via Android's native `Intent.ACTION_SEND` through `share_plus` . This complies fully with Google Play's Scoped Storage policy without requiring dangerous broad-storage permissions.

7. PRODUCTION RELEASE SIGNING

1. Generate an industry-standard RSA 2048-bit release keystore:

```
keytool -genkey -v -keystore android/app/release-keystore.jks -keyalg RSA -keysize 2048 -validity 10000 -alias minipos
_key
```

2. Configure `android/key.properties` with your credentials:

```
storePassword=your_keystore_password
keyPassword=your_key_password
keyAlias=minipos_key
storeFile=release-keystore.jks
```

8. COMPILING OPTIMIZED PRODUCTION BINARIES

Build minified, tree-shaken release artifacts ready for distribution:

```
# Android App Bundle (Production optimized for Google Play)
flutter build appbundle --release --obfuscate --split-debug-info=build/app/outputs/symbols

# Standalone Universal APK (For Direct Client Distribution / Gumroad)
flutter build apk --release
```